

Berliner Hochschule für Technik

Fachbereich VII: Elektrotechnik - Mechatronik - Optometrie

Bachelorarbeit

Vergleich verschiedener Dateisysteme für den SD-Karten-Einsatz in eingebetteten Systemen

Autor: Michael Kolorz

Matrikelnummer: 100359

Studiengang: Elektrotechnik (B.Eng.)

Schwerpunkt: Kommunikationstechnik

Erstprüfer: Prof. Dr. Peter Gober

Zweitprüfer: Prof. Dr. David Dietrich

Semester: Sommersemester 2026

Abgabedatum: 1. Oktober 2026

Inhaltsverzeichnis

1	Einleitung	2
2	SD	2
3	Dateisysteme	3
3.1	FatFS	3
3.1.1	Einleitung	3
3.1.2	Implementierung	3
	Quellenverzeichnis	4

1 Einleitung

Blah blub

2 SD

Tabelle 1: Pin-Belegung einer Micro-SD-Karte im SPI-Modus

Pin	SD-Modus	SPI-Modus	Beschreibung (SPI)	STM32 G431 Pin
1	DAT2	<i>Nicht genutzt</i>	Frei lassen (oder Pull-up)	–
2	DAT3 / CD	CS	Chip Select (Low-aktiv)	GPIO (z. B. PA4)
3	CMD	DI	Data In (Dateneingang)	PA7 (MOSI)
4	VDD	VDD	Stromversorgung (3,3 V)	3,3 V
5	CLK	SCLK	Taktleitung	PA5 (SCK)
6	VSS	VSS	Masse	GND
7	DAT0	DO	Data Out (Datenausgang)	PA6 (MISO)
8	DAT1	<i>Nicht genutzt</i>	Frei lassen (oder Pull-up)	–

Tabelle 2: Pin-Belegung einer Standard-SD-Karte im SPI-Modus

Pin	SD-Modus	SPI-Modus	Beschreibung (SPI)	STM32 G431 Pin
1	DAT3 / CD	CS	Chip Select (Low-aktiv)	GPIO (z. B. PA4)
2	CMD	DI	Data In (Dateneingang)	PA7 (MOSI)
3	VSS1	VSS	Masse	GND
4	VDD	VDD	Stromversorgung (3,3 V)	3,3 V
5	CLK	SCLK	Taktleitung	PA5 (SCK)
6	VSS2	VSS	Masse	GND
7	DAT0	DO	Data Out (Datenausgang)	PA6 (MISO)
8	DAT1	<i>Nicht genutzt</i>	Frei lassen	–
9	DAT2	<i>Nicht genutzt</i>	Frei lassen	–

Tabelle 3: Anschlussbelegung der MicroSD-Karte am STM32G431 im SPI-Modus

MicroSD-Pin	Signalname	Verbindung am STM32G431 / Platine
Pin 1	DAT2	3.3V über internen Pull-up
Pin 2	CS	PA9
Pin 3	DI (MOSI)	PA7
Pin 4	VDD	3.3V Hauptversorgung
Pin 5	SCK	PA5
Pin 6	VSS	GND
Pin 7	DO (MISO)	PA6
Pin 8	DAT1	3.3V über internen Pull-up

3 Dateisysteme

3.1 FatFS

3.1.1 Einleitung

FatFs ist ein plattformunabhängiges, quelloffenes Software-Modul zur Implementierung von FAT/exFAT-Dateisystemen in ressourcenbeschränkten, eingebetteten Systemen. Es wurde von dem Entwickler ChaN[3] entworfen und zeichnet sich durch einen extrem geringen Speicherbedarf sowohl im Programm- als auch im Arbeitsspeicher aus. Das Modul ist vollständig in ANSI C (C89) geschrieben. Dadurch ist es hardwareunabhängig auf praktisch jedem gängigen Mikrocontroller (z. B. ARM, AVR, PIC oder STM32) ohne Änderungen des Codes direkt lauffähig.

Die Architektur trennt das Dateisystem strikt in zwei logische Schichten: Das Application Interface stellt der Anwendung abstrakte, bekannte Funktionen zur Datei- und Verzeichnisverwaltung wie `f_open`, `f_read`, `f_write` und `f_close` zur Verfügung, während das Media Access Interface (Disk I/O) die Hardware vollkommen vom Dateisystem entkoppelt.

3.1.2 Implementierung

```
1  DSTATUS disk_initialize(BYTE pdrv);
2  DSTATUS disk_status(BYTE pdrv);
3  DRESULT disk_read(...);
4  DRESULT disk_write(...);
5  DRESULT disk_ioctl(...);
```

Quellcode 1: Funktionen, die in der `diskio.cpp` implementiert werden mussten

`ffconf.h`

Die Datei `ffconf.h` ist die zentrale Konfigurationsdatei des FatFs-Moduls. Über den C-Präprozessor lässt sich der Funktionsumfang auf die Hardware-Ressourcen des Mikrocontrollers zuschneiden.

- **Anpassbarer Funktionsumfang:**

- `FF_FS_READONLY`: Schaltet das Schreiben komplett ab, was massiv Flash-Speicher einspart.
- `FF_FS_MINIMIZE`: Entfernt gezielt ungenutzte APIs wie Verzeichnisoperationen (`f_mkdir`, `f_opendir`) oder Suchfunktionen.
- `FF_USE_MKFS`: Aktiviert oder deaktiviert die Formatierungsfunktion `f_mkfs`.

- **Namensräume und Lokalisierung:**

- `FF_USE_LFN`: Schaltet die Unterstützung für lange Dateinamen frei.
- `FF_CODE_PAGE`: Legt die Codepage fest, um Sonderzeichen korrekt darzustellen.

- **Speicher- und Laufwerksoptionen:**

- `FF_VOLUMES`: Definiert die Anzahl der logischen Laufwerke, die gleichzeitig verwaltet werden können.
- `FF_MAX_SS` & `FF_MIN_SS`: Steuert die Sektorgröße, was wichtig für Medien mit Sektoren größer als 512 Byte ist.

- **Systemeinstellungen:**

- `FF_FS_TINY`: Reduziert den RAMbedarf pro Datei, indem interne Puffer ausgelagert werden.
- `FF_FS_EXFAT`: Aktiviert die Unterstützung von exFAT für moderne Speichermedien über 32 GB.

Da der verwendete STM32-G431 für einen Mikrocontroller über verhältnismäßig viele Ressourcen verfügt, konnten viele Zusatzoptionen wie die Unterstützung für das exFAT-Dateisystem kompiliert werden.

Quellenverzeichnis

- [1] Ababei, C. (2013). *Lecture 12: SPI and SD cards*. Vorlesungsskript, EE-379 Embedded Systems and Applications, University at Buffalo. Abgerufen von https://www.dejazzzer.com/ee379/lecture_notes/lec12_sd_card.pdf (Besucht am 6. Juli 2026).
- [2] SD Group Matsushita Electric Industrial Co., Ltd. (Panasonic) (2006). *SD Specifications Part 1: Physical Layer Simplified Specification, Version 2.00*. Technische Spezifikation, SD Card Association. Abgerufen von https://users.ece.utexas.edu/~valvano/EE345M/SD_Physical_Layer_Spec.pdf (Besucht am 8. Juli 2026).
- [3] ChaN (2024). *FatFs - Generic FAT Filesystem Module*. Software-Dokumentation, Electronic Lives Manufacturing. Abgerufen von <https://elm-chan.org/fsw/ff/> (Besucht am 23. Juli 2026).
- [4] Microchip Technology Inc. (2024). *How to interface a microSD media with Microchip SD/MMC Card Readers*. Support Knowledge Base. Abgerufen von <https://support.microchip.com/s/article/How-to-interface-a-microSD-media-with-Microchip-SD-MMC-Card-Readers> (Besucht am 8. Juli 2026).
- [5] Michael Eiler, *C++20: Building a Thread-Pool With Coroutines* <https://blog.eiler.eu/posts/20210512/>